



AMAZON FRESH

MySQL Analytics Project

Database Design, Querying & Insights

Submitted By	S. Seshang
Database	amazon
Tool	MySQL Workbench
Total Tasks	14 Tasks Covered
Topics	DDL, DML, Joins, Subqueries, Normalization

Exploring Data. Delivering Insights. Powering Decisions.

Task 1: ER Diagram & Table Relationships

Create an ER diagram for the Amazon Fresh database to understand relationships between tables (Customers, Products, Orders).

The ER diagram was designed to show the relationships between Customers, Products, Orders, Order_Details, Suppliers, and Reviews tables. Primary keys and foreign keys were identified for each table.

SQL Query

```
-- Task 1: ER Diagram (conceptual representation)
-- Tables: Customers, Products, Orders, Order_Details, Suppliers, Reviews
-- Relationships:
-- Customers (CustomerID PK) -> Orders (CustomerID FK)
-- Products (ProductID PK) -> Order_Details (ProductID FK)
-- Suppliers (SupplierID PK) -> Products (SupplierID FK)
-- Orders (OrderID PK) -> Order_Details (OrderID FK)
-- Products (ProductID PK) -> Reviews (ProductID FK)
```

Task 2: Primary Keys & Foreign Keys

Identify the primary keys and foreign keys for each table and describe their relationships.

Each table was analyzed and primary/foreign key constraints were identified.

SQL Query

```
-- Customers: CustomerID (PK)
-- Products: ProductID (PK), SupplierID (FK -> Suppliers)
-- Orders: OrderID (PK), CustomerID (FK -> Customers)
-- Order_Details: OrderDetailID (PK), OrderID (FK -> Orders), ProductID (FK -> Products)
-- Suppliers: SupplierID (PK)
-- Reviews: ReviewID (PK), ProductID (FK -> Products), CustomerID (FK -> Customers)
```

Task 3: Basic SELECT Queries

Retrieve all customers from a specific city. Fetch all products under the "Fruits" category.

SQL Query

```
-- Retrieve all customers from a specific city
SELECT * FROM amazon.customers
WHERE city = 'Meganfort';

-- Fetch all products under the 'Fruits' category
SELECT ProductName, Category FROM amazon.products
WHERE Category = 'Fruits';
```

Task 4: DDL Statements – Recreating Customers Table

Write DDL statements to recreate the Customers table with constraints: CustomerID as PK, Age > 18 (NOT NULL), and unique constraint for Name.

SQL Query

```
-- CustomerID as primary key
ALTER TABLE amazon.customers
ADD PRIMARY KEY (CustomerID);

-- Ensure Age cannot be null and must be greater than 18
SELECT Age FROM amazon.customers
WHERE age > 18;

-- Add unique constraint for Name
ALTER TABLE amazon.customers
ADD UNIQUE INDEX `Name_UNIQUE` (`Name` ASC) VISIBLE;
```

Task 5: INSERT Statements – Products Table

Insert 3 new rows into the Products table.

SQL Query

```
INSERT INTO amazon.products VALUES
('4', 'table', 'wood', 'sub-wood', 111, 123, '100'),
('5', 'fan', 'electronics', 'sub-electronics', 222, 234, '101'),
('6', 'car', 'motor', 'sub-motor', 333, 345, '102');

SELECT * FROM amazon.products;
```

Task 6: UPDATE – Stock Quantity

Update the stock quantity of a product where ProductID matches a specific ID.

SQL Query

```
UPDATE amazon.products
SET StockQuantity = 777
WHERE ProductName = 'table';
```

Task 7: DELETE – Supplier by City

Delete a supplier from the Suppliers table where their city matches a specific value.

SQL Query

```
DELETE FROM amazon.suppliers
WHERE city = 'South Debra';

SELECT * FROM amazon.suppliers;
```

Task 8: SQL Constraints

Add a CHECK constraint to ensure ratings in the Reviews table are between 1 and 5. Add a DEFAULT constraint for PrimeMember column (default: "No").

SQL Query

```
-- CHECK constraint for ratings
ALTER TABLE amazon.reviews
ADD CONSTRAINT CHECK (Rating BETWEEN 1 AND 5);

-- DEFAULT constraint for PrimeMember
ALTER TABLE amazon.customers
ALTER PrimeMember SET DEFAULT 'No';
```

Task 9: WHERE, HAVING, GROUP BY & ORDER BY

Write queries using WHERE, HAVING, GROUP BY, and ORDER BY clauses.

SQL Query

```
-- WHERE clause: orders placed after 2024-01-01
SELECT * FROM amazon.orders
WHERE OrderDate > '2024-01-01';

-- HAVING clause: products with average rating > 4
SELECT p.ProductName, r.ProductId, AVG(Rating) AS avg_rating
FROM amazon.reviews AS r
JOIN amazon.products AS p ON p.ProductID = r.ProductID
GROUP BY p.ProductName, r.ProductID
HAVING avg_rating > 4
ORDER BY avg_rating ASC;

-- GROUP BY & ORDER BY: rank products by total sales
SELECT p.ProductName, p.ProductID,
       SUM(o.Quantity * o.UnitPrice - o.Discount) AS total_sales
FROM amazon.order_details AS o
JOIN amazon.products AS p ON p.ProductID = o.ProductID
GROUP BY p.ProductName, p.ProductID
ORDER BY total_sales DESC;
```

Task 10: Top Customers by Total Spending

Calculate total spending per customer, rank them, and identify customers who spent more than ₹5,000.

SQL Query

```
-- Calculate each customer's total spending
SELECT c.Nameess, o.CustomerID,
       SUM(o.OrderAmount + o.DeliveryFee - DiscountApplied) AS total_spending
FROM amazon.orders AS o
JOIN amazon.customers AS c ON c.CustomerID = o.CustomerID
GROUP BY c.Nameess, o.CustomerID
ORDER BY total_spending DESC;

-- Rank customers using DENSE_RANK()
SELECT c.Nameess, o.CustomerID,
       SUM(o.OrderAmount + o.DeliveryFee - DiscountApplied) AS total_spendings,
       DENSE_RANK() OVER(ORDER BY SUM(o.OrderAmount + o.DeliveryFee - DiscountApplied) DESC) AS rankk
FROM amazon.orders AS o
JOIN amazon.customers AS c ON c.CustomerID = o.CustomerID
GROUP BY c.Nameess, o.CustomerID;

-- Customers who spent more than Rs.5000
SELECT c.Nameess, o.CustomerID,
       SUM(o.OrderAmount + o.DeliveryFee - DiscountApplied) AS total_spending
FROM amazon.orders AS o
JOIN amazon.customers AS c ON c.CustomerID = o.CustomerID
GROUP BY c.Nameess, o.CustomerID
HAVING total_spending > 5000
ORDER BY total_spending ASC;
```

Task 11: JOIN Queries – Revenue, Orders & Suppliers

Join Orders and OrderDetails to calculate total revenue per order. Identify customers with most orders in a time period. Find supplier with most products in stock.

SQL Query

```
-- Total revenue per order
SELECT o.OrderID, SUM(o2.Quantity * o2.UnitPrice - o2.Discount) AS revenue
FROM amazon.orders AS o
JOIN amazon.order_details AS o2 ON o.OrderID = o2.OrderID
GROUP BY o.OrderID
ORDER BY revenue DESC;

-- Customers with most orders on a specific date
SELECT c.Nameess, c.CustomerID, o.OrderDate, COUNT(o.OrderID) AS total_orders
FROM amazon.orders AS o
JOIN amazon.customers AS c ON c.CustomerID = o.CustomerID
WHERE OrderDate = '2025-01-01'
GROUP BY c.Nameess, c.CustomerID
ORDER BY total_orders DESC;

-- Supplier with most products in stock
SELECT SupplierID, SUM(StockQuantity) AS total
FROM amazon.products
GROUP BY SupplierID
ORDER BY total DESC;
```

Task 12: Normalization to 3NF – Products Table

Normalize the Products table to 3NF by separating product categories and subcategories into new tables and creating foreign keys.

SQL Query

```
-- Create ProductCategory table
CREATE TABLE amazon.ProductCategory (
  special_id   VARCHAR(100),
  product_id   VARCHAR(100),
  product_category VARCHAR(100),
  package_color VARCHAR(100)
);

-- Insert sample data into ProductCategory
INSERT INTO amazon.ProductCategory VALUES
('1','0006853b-...','Bakery','black'),
('2','0219aafa-...','Dairy','white'), ...;

-- Create SubCategory table
CREATE TABLE amazon.SubCategory (
  sub_category_id VARCHAR(100),
  special_id   VARCHAR(100),
  product_id   VARCHAR(100),
  sub_category VARCHAR(100)
);

-- Insert sample data into SubCategory
INSERT INTO amazon.SubCategory VALUES
('1a','1','0006853b-...','sub-bakery'),
('2a','2','0219aafa-...','sub-dairy'), ...;

-- Foreign key relationships maintained between tables
```

Task 13: Subqueries – Top Products & Inactive Customers

Identify the top 3 products based on sales revenue. Find customers who haven't placed any orders yet.

SQL Query

```
-- Top 3 products by sales revenue
SELECT p.ProductID, p.ProductName,
       SUM(o.Quantity * o.UnitPrice - o.Discount) AS total_revenue
FROM amazon.order_details AS o
JOIN amazon.products AS p ON p.ProductID = o.ProductID
GROUP BY p.ProductID, p.ProductName
ORDER BY total_revenue DESC
LIMIT 3;

-- Customers who haven't placed any orders
SELECT c.Name, o.CustomerID
FROM amazon.orders AS o
JOIN amazon.customers AS c ON c.CustomerID = o.CustomerID
WHERE o.CustomerID IS NULL;
```

Task 14: Actionable Insights

Which cities have the highest concentration of Prime members? What are the top 3 most frequently ordered categories?

SQL Query

```
-- Cities with highest concentration of Prime members
SELECT City, COUNT(PrimeMember)
FROM amazon.customers
WHERE PrimeMember = 'Yes'
GROUP BY City
ORDER BY COUNT(PrimeMember) DESC
LIMIT 1;

-- Top 3 most frequently ordered categories
SELECT Category, COUNT(Category) AS total_quantity_ordered
FROM amazon.products
GROUP BY Category
ORDER BY total_quantity_ordered DESC
LIMIT 3;
```